SDN-based IDS + IPS

Concepts, detection logic, dashboard explained, and a one-page command sheet

> **What this project does, in five lines**
> A small office network runs on one laptop. 13 computers are connected to one switch. Two of the computers (h1 and h10) attack the servers on purpose. A program called the controller watches the traffic and decides if a computer is attacking. If it is, the controller tells the switch to drop all packets from that computer. A web page (the dashboard) shows everything that is happening, live.

The two main words

	Meaning	In this project
IDS	**Intrusion Detection System.** It looks at traffic and decides: is this an attack? It only *detects* and writes an alert. It does not stop anything.	`controller/ids_engine.py` (the rules) and the `[IDS ALERT]` log lines.
IPS	**Intrusion Prevention System.** It does everything the IDS does, and then it *stops* the attack automatically.	`controller/ids_controller.py`. After an alert it installs a DROP rule in the switch and removes it later by itself.

Short version: IDS = notice the attack. IPS = notice the attack and block it. This project does both, so it is an IDS + IPS.

What is inside this PDF

1 Words you need to know (glossary)
2 How the pieces fit together (architecture)
3 The IDS and IPS logic, start to end
4 The seven detection rules
5 The rules inside the switch (priorities)
6 The dashboard (telemetry panel), part by part
7 The lab network and the project files
8 Questions you may be asked, with simple answers
9 **One-page command sheet** (what to run, in which terminal)

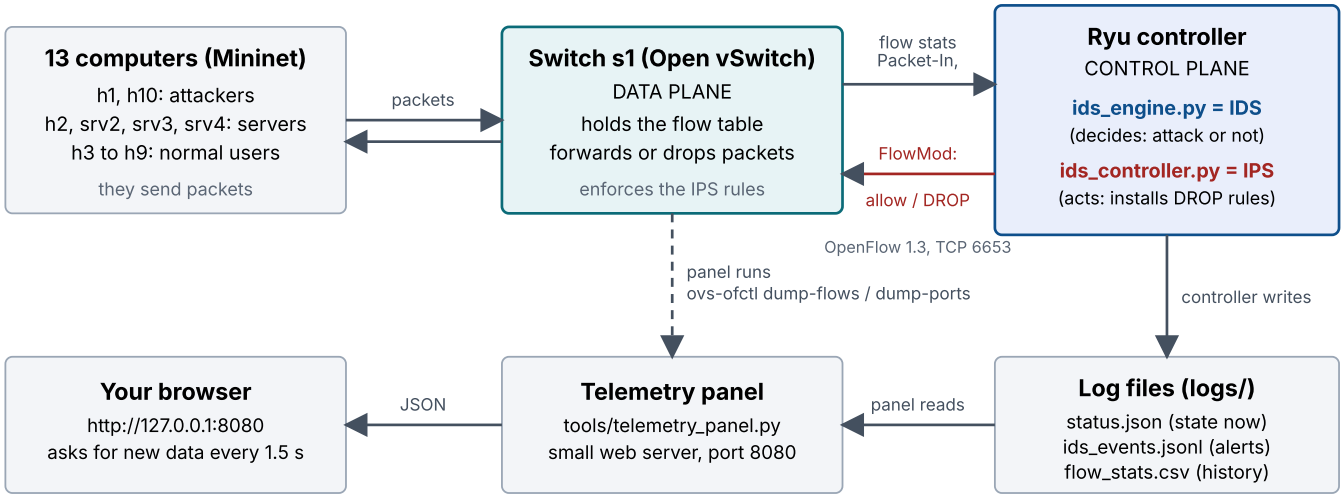
> **Read this once before presenting**
> The IDS part is the *deciding*. The IPS part is the *acting*. When someone asks "where is the IDS and where is the IPS?", point to the engine file for the IDS and to the controller file (DROP rule) for the IPS.

1. Words you need to know

Word	Plain meaning
Packet	A small piece of data sent over a network. Every web page, message or file is split into many packets.
IP address	The address of a computer on the network, for example `10.0.0.2`. It tells packets where to go.
Port	A number that says which program on a computer should get the packet. Port 80 is a web server, port 22 is SSH, port 3306 is a database.
TCP	The most common way to send data. A connection must be opened first, then data flows, then the connection is closed.
SYN	The very first packet that asks to open a TCP connection. One SYN = one new connection request.
TCP flags	Small on/off marks inside a TCP packet (SYN, ACK, FIN, RST, PSH, URG). Normal traffic uses only certain combinations. Other combinations are not normal and are used by scanners.
Connection / flow	One conversation between two computers: source IP + source port + destination IP + destination port. Also called a 5-tuple flow when the protocol is included.
pps	Packets per second. How many packets a computer sends each second.
Switch	The device all computers plug into. It receives each packet and sends it out of the correct port.
Open vSwitch (OVS)	A software switch. Here it is the switch named `s1`. It runs on your laptop.
Mininet	A tool that creates fake computers (h1, h2 ...) and a fake network on one laptop, so no real cables are needed.
SDN	Software Defined Networking. The decisions ("what do I do with this packet?") are made by a separate program (the controller), not inside the switch.
Data plane	The part that moves packets: the switch (OVS) and the computers.
Control plane	The part that makes decisions: the controller (Ryu). It sends rules to the switch.
Ryu	A Python program (framework) used to write the controller.
OpenFlow 1.3	The language the controller and the switch use to talk to each other. It runs over TCP port `6653`.
Flow table / flow rule	The switch's list of rules. Each rule has a **match** (which packets), an **action** (what to do: forward or drop) and a **priority** (higher number is checked first).
Packet-In	A message from the switch to the controller that says "I got a packet and I have no rule for it. What should I do?"
FlowMod	A message from the controller to the switch that adds or removes a rule.
Barrier	A message that makes the switch confirm "I finished applying all the rules you sent before this." It is used to measure the real block time.
Idle timeout	A rule is deleted automatically if no packet matches it for this many seconds.
Whitelist (protected hosts)	A list of trusted IP addresses (the servers and clients). The IDS never raises an alert for them. Set in `ids_rules.json`.
Quarantine	Dropping everything that comes in from one physical switch port, whatever its IP address says.

2. How the pieces fit together

There are three groups of parts: the **network** (computers and the switch), the **controller** (the brain), and the **dashboard** (what you look at).



Why two planes?

The switch is fast but has no brain. It only follows its rules. The controller has the brain but is slower. So:

- The first packet of every new connection goes to the controller, so the controller can check it.
- If it is fine, the controller puts a rule into the switch. The rest of that connection never reaches the controller. This keeps the controller from being overloaded.
- If it is an attack, the controller puts a DROP rule into the switch. The switch then throws those packets away by itself.

Why the start order matters (Terminal 1 first, then Terminal 2)

When Mininet starts, it cleans up old processes and kills any running `ryu-manager`. So the network must be started first, and the controller after it. If you start the controller first, Mininet kills it. The switch is set to `fail-mode secure`, which means it keeps its rules and waits for the controller to connect.

The three data sources the dashboard reads

Source	Written by	What is in it
logs/status.json	Controller, every 3 s and after every change	Which switches are connected, which hosts are blocked and why, per-host stats, timeout settings
logs/ids_events.jsonl	Controller, on every event	One line per ALERT, BLOCK_INSTALLED and UNBLOCKED event
logs/flow_stats.csv	Controller, every 3 s	History of flows, packets/s and bytes
Switch (live)	Panel runs `ovs-ofctl dump-flows s1`, `dump-ports s1` and `show s1`	The real flow table and the real packet counters of every port

3. The IDS and IPS logic, start to end

This is exactly what happens from the moment the system starts until a blocked host is released.

1. **Start-up.** The switch connects to the controller on port 6653. The controller deletes every old rule in the switch. Then it installs two rules that send traffic to the controller: priority **0** (any packet with no other rule) and priority **5** (any TCP packet with no faster rule).
2. **A new connection begins.** A computer sends a SYN packet. The switch has no rule for it yet, so it sends the packet to the controller as a **Packet-In**.
3. **The controller learns where the sender is.** It remembers "this MAC address is on this switch port". It uses that to know which port the packet should leave from.
4. **Quick checks.** If the sender is already blocked, or its port is quarantined, the packet is ignored. If the sender is on the protected list (whitelist), the IDS skips all checks and the packet is allowed.
5. **The IDS decides.** `observe_tcp()` in `ids_engine.py` runs these checks in order: `multi_ip_spoof`, `port_flood`, `invalid_flags`, `conn_flood`, `port_scan`. Each check keeps a short history (a time window) per sender and compares it to a limit.
6. **No alert: allow it.** The controller installs **two priority-20 rules**, one for each direction of the connection, with a 10-second idle timeout. It then sends the packet on. All later packets of this connection are handled by the switch alone. The controller never sees them.
7. **Alert: the IPS acts.** `_react()` in `ids_controller.py` writes `[IDS ALERT]` to the log and the event file. It then installs a DROP rule: **priority 100** that drops every packet from that source IP, or **priority 200** that drops everything entering through that switch port (plus a priority-100 drop on the source MAC). The rule is installed with a 30-second idle timeout.
8. **Confirm.** The controller sends a **Barrier** request. When the switch answers, the DROP rule is certainly active. The controller logs `[IDS RESPONSE] MITIGATED` and records three times: detection time, response time and total time.

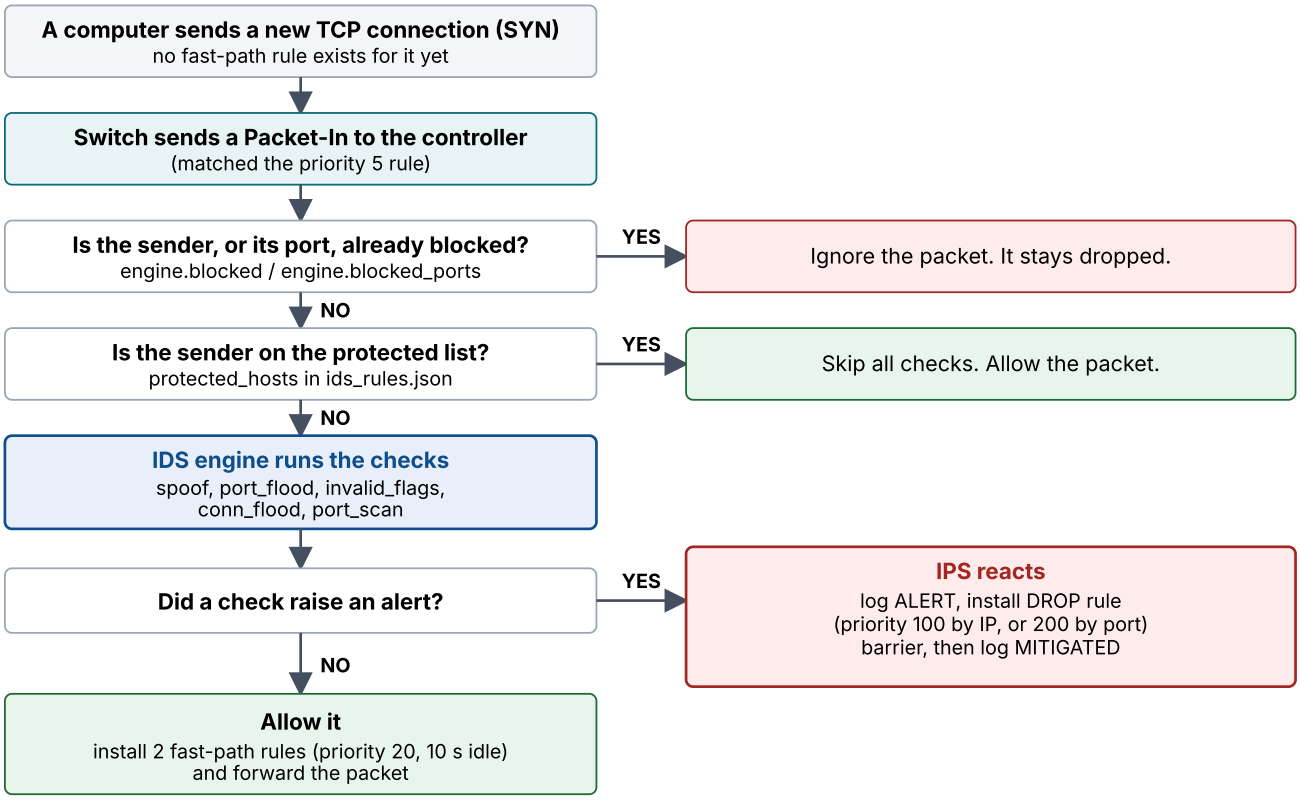


Figure: what the controller does with every new TCP connection.

3. (continued) Catching attacks the controller never sees

9. **The problem.** After step 6, packets of an allowed connection do not reach the controller any more. So an attacker who opens a few connections and then sends a huge number of packets, or who keeps many connections open, would not trigger the Packet-In checks.

10. **The fix: counting inside the switch.** Every **3 seconds** (`stats_interval`) the controller asks the switch for the counters of all priority-20 rules (`OFPFlowStatsRequest`). For each source IP it works out two numbers:
    - **flows** = how many connections that source has open right now,
    - **pps** = packets added to its counters since the last poll, divided by the seconds between polls.

    `evaluate_stats()` compares them to the limits: `concurrent_flows` (40 or more flows) or `packet_rate` (3000 or more pps). An alert uses the same IPS reaction as step 7. If both are true, `concurrent_flows` is the one reported.

11. **Self-healing (release).** The DROP rule has a 30-second idle timeout. When no packet has matched it for 30 seconds, the switch deletes it and sends a **FlowRemoved** message. The controller then forgets the block and the old counters, logs `[IDS RECOVERY] UNBLOCKED`, and the host is allowed again. If the attacker keeps sending, packets keep matching the rule, so the block stays.

12. **Everything is written down.** Alerts go to `ids_events.jsonl`, history to `flow_stats.csv`, and the current state to `status.json`. The dashboard reads these files and the live switch.

Two detection paths, one reaction

	Path A: Packet-In (real time)	Path B: flow-counter polling (every 3 s)
Sees	The first packet of every new connection	Counters of connections that are already open
Catches	invalid_flags, port_scan, conn_flood, multi_ip_spoof, port_flood	concurrent_flows, packet_rate
Speed	Within milliseconds	Within one polling interval (up to about 3 s)
Why it exists	Stops scans and connection floods immediately	Catches heavy traffic without sending all that traffic to the controller

Both paths end in the same place: `_react()` installs the DROP rule.

IDS and IPS, separated clearly

Question	IDS	IPS
Job	Decide if traffic is an attack	Stop the traffic that was judged an attack
File	`ids_engine.py`	`ids_controller.py`
Output	`Alert` objects, `[IDS ALERT]` log lines, `ids_events.jsonl`	DROP rules in the switch, `[IDS RESPONSE] MITIGATED`, `[IDS RECOVERY]`
If removed	Nothing would be judged	You would still get alerts, but attacks would continue

> **Why the engine is a separate file**
> `ids_engine.py` has no Ryu code in it. It only takes packet details in and returns alerts. That is why it can be tested alone, without the switch or controller.

4. The seven detection rules

All limits come from `controller/ids_rules.json`. The controller reloads this file by itself every 3 seconds, so changes apply without a restart.

Rule	It fires when ...	Limit	Path	Block type	Test key
invalid_flags	One source sends packets with a flag combination that normal software never uses (NULL, XMAS, SYN+FIN, SYN+RST, or FIN without ACK).	3 packets in 10 s	A	IP drop, prio 100	1
port_scan	One source sends connection requests to many different destination ports. A scan is looking for open ports.	8 distinct ports in 10 s	A	IP drop, prio 100	2
multi_ip_spoof	The same MAC address sends packets with many different source IP addresses. A real computer has one IP address.	more than 2 IPs in 4 s	A	Port quarantine, prio 200 (+ MAC drop, prio 100)	3

conn_flood	One source opens connections far faster than a person or program normally does.	20 new connections in 3 s	A	IP drop, prio 100	4
port_flood	One switch port sends a very high number of SYNs, whatever source IPs they use. This catches floods that use random IP addresses.	35 SYNs in 3 s	A	Port quarantine, prio 200	(automatic)
concurrent_flows	One source keeps too many connections open at the same time, even if it opens them slowly.	40 open flows	B	IP drop, prio 100	5
packet_rate	One source sends too many packets per second (a volumetric flood). It can do this with only a few connections.	3000 pps	B	IP drop, prio 100	6

Check order and what it means

- `multi_ip_spoof` and `port_flood` are checked first. If one fires, the engine stops there and the whole **port** is quarantined.
- Then `invalid_flags`, `conn_flood` and `port_scan` are checked. More than one can fire at the same time. All are logged and one block is installed.
- A source that is already blocked is not checked again.

IP drop versus port quarantine

	IP drop (priority 100)	**Port quarantine (priority 200)**
Matches	Every IPv4 packet whose source IP is the attacker's	Every packet entering through that switch port
Used when	The attacker uses one real IP address	The attacker changes IP addresses, so blocking an IP would not help
Extra	None	Also drops the attacker's MAC address at priority 100

Who is never blocked

The servers (10.0.0.2, 10.0.0.10, 10.0.0.11, 10.0.0.12) and the normal clients (10.0.0.3 to 10.0.0.9) are in `protected_hosts`. The two attackers, h1 (10.0.0.1) and h10 (10.0.0.13), are not. This is why normal users keep working during an attack.

> **Two things to know before the demo**
>
> 1) The 3-second polling means a `packet_rate` or `concurrent_flows` block can appear up to about 3 seconds after the flood starts. 2) The block is released only after the attacker has stopped for 30 seconds. To start the next test at once, press **8** (clear blocks) in Terminal 4.

5. The rules inside the switch

The switch checks rules from the highest priority number to the lowest. The first rule that matches decides what happens. You see these rules at the bottom of the dashboard in the "OVS flow table" box.

Priority	Name on dashboard	What it matches	Action	Created by
200	ips: port quarantine	All packets coming in through one port	DROP	IPS, on alert
100	ips: block (ip / mac)	All packets from one source IP (or MAC)	DROP	IPS, on alert
20	tcp connection (fast path)	One exact connection (source IP, destination IP, source port, destination port)	Forward out of the right port	Controller, after a clean check. Idle timeout 10 s
5	new tcp, ids inspect	Any TCP packet without a priority-20 rule	Send to controller	Controller, at start-up
1	l2 forwarding	Other traffic (such as ARP and ping) learned by MAC address	Forward. Idle timeout 30 s	Controller, when learned
0	table-miss to controller	Anything not matched above	Send to controller	Controller, at start-up

> **Why this order works**
> A DROP rule (100 or 200) has a higher number than the fast-path rule (20). So once a source is blocked, the DROP rule wins even if its old fast-path rules are still in the table.

6. The dashboard (telemetry panel), part by part

The panel is `tools/telemetry_panel.py`. It is a small web server using only Python's standard library. Open `http://127.0.0.1:8080`. The page asks the server for new data every **1.5 seconds** at `/api/telemetry`. The page is loaded once when the panel starts, so after any change restart Terminal 3 and press **Ctrl+Shift+R** in the browser.

How the panel gets its data

1. It reads the three files in `logs/` (see section 2).
2. It runs three read-only commands on the switch: `ovs-ofctl dump-flows s1` (the rules), `dump-ports s1` (packet counters of every port) and `show s1` (which port number belongs to which host).
3. It puts everything into one JSON answer. The browser calculates rates (packets per second) by comparing two answers in a row.

That is why the panel must run with `sudo`: reading the switch needs it.

Part of the page	What it shows	Where the numbers come from
Top bar: controller	Green "up" means the controller is alive. Red "down" means it stopped.	`status.json` was updated in the last 15 s (or 4 poll intervals if longer).
Top bar: switch s1	Green "connected" means the switch is attached to the controller.	Switch id 1 is listed in `status.json`.
Red banner "INTRUSION"	Appears only while at least one host is blocked. Names the host, IP and the rule that fired.	`blocked` list in `status.json`.
Stat strip: alerts	How many ALERT events are in the recent log (last 120 events).	`ids_events.jsonl`
Stat strip: blocked	How many hosts are blocked right now, and which.	`status.json`
Stat strip: network packets/s	Total packets sent into the switch by all hosts each second.	Sum of the receive counters of all 13 switch ports (`dump-ports`).
Stat strip: throughput	Total bytes per second sent by all hosts.	Port byte counters.
Stat strip: live connections	Number of allowed TCP connections open now. Below it: number of client-to-service pairs.	Priority-20 rules in the flow table, both directions merged.
Stat strip: dropped by IPS	Total packets thrown away by DROP rules. Below it: number of drop rules and total rules.	Packet counters of the priority-100 and 200 rules.

Part of the page	What it shows and how to read it	Where the numbers come from
Traffic graph	One line per host: packets per second that host *sent*. A tall spike is a flood. A line that falls back to zero after a spike means the block worked. The orange dashed line is the packet_rate limit (3000). **linear** shows true size. **log** compresses large values so small hosts stay visible next to a big flood. It remembers the last 90 readings (about 2 minutes); reloading the page clears it.	Port counters from `dump-ports`, one reading every 1.5 s.
Hosts table	All 13 hosts. **port**: switch port number. **sent/s**: packets per second the host sends. **recv/s**: packets per second delivered to the host. **pkts sent / pkts recv**: totals since the switch started. **conns**: connections that involve this host. **dropped**: packets from this host thrown away by DROP rules. **state**: see below.	Port counters, flow table and `status.json`.
Live connections	Who talks to whom. Each row is client, server and service (port). **conns**: how many connections. **pkts/s**: current packet rate. **packets, bytes**: totals. **idle**: seconds since the last packet. **status**: green ALLOWED, or red BLOCKED. A red row at the top shows a blocked source and how many of its packets were dropped.	Priority-20 rules (their packet counters are compared between two readings).
Network totals - history	Three small charts: packets/s, bytes/s and active TCP flows, with the highest value ("peak") shown above each chart.	`flow_stats.csv`, written every 3 s by the controller.
Active blocks	Every blocked host with the rule that fired, the type of drop (IP or port), the detail text, and "idle ttl". The ttl is an estimate of the 30-second release timer. The switch only releases the block after 30 seconds *without* matching packets.	`status.json`
Traffic by service	Bars showing which destination ports get the most packets per second (22 ssh, 80 http, 3306 mysql, 5000 app and so on). During a flood, the attacked port shows a very long bar.	Priority-20 rules grouped by server port.
Activity feed: all traffic	A running list, newest first. **OPEN**: new connections appeared (several of the same kind are merged, for example "x3"). **CLOSE**: connections ended; this shows about 10 seconds after the last packet because of the 10 s idle timeout. **ALERT**, **BLOCK** and **RECOVER** events from the IDS/IPS are mixed in.	Browser compares the flow table between two readings, plus `ids_events.jsonl`.
Activity feed: ids events	Only the IDS/IPS events: ALERT (attack detected, with detect time), BLOCK (rule installed, with response time in ms), RECOVER (block removed).	`ids_events.jsonl`

OVS flow table	The real rule list of the switch, highest priority first. Buttons filter: **all**, **forward** (FWD), **drop** (DROP), **to controller** (CTRL). The role column names each priority (section 5). The line above the table shows total packets and bytes matched.	`ovs-ofctl dump-flows s1`
Detection rules strip	The seven rules with their current limits, plus the block idle timeout, the poll interval and how many hosts are whitelisted. A struck-through rule is disabled.	`controller/ids_rules.json`
Footer	Time of the last update and "telemetry age" (seconds since the controller last wrote its status). Any error text, for example when the switch cannot be read, is shown here in amber.	Panel and `status.json`

Host state labels

Label	Meaning
BLOCKED	The IPS is dropping this host's traffic now.
ACTIVE **/ attacker**	An attacker host that is sending traffic now / not sending.
SERVING **/ PROT**	A protected server that is handling traffic now / waiting.
UP **/ idle**	A normal client that is sending traffic now / not sending.

> **What a good demo looks like on the dashboard**
> Before the attack: all 13 rows show small numbers, many ALLOWED rows, no red. During the attack: one line spikes in the graph, the attacker's row turns ACTIVE, its target port gets the longest bar. After detection: red banner, BLOCKED row, an ALERT and a BLOCK line in the feed, "dropped by IPS" starts counting. Throughout: the normal clients stay UP and ALLOWED. That shows no collateral damage.

7. The lab network and the project files

The 13 computers

Name	IP address	Role	Services (ports)	Switch port
h1	10.0.0.1	Attacker 1	none	1
h2	10.0.0.2	App server	22, 80, 443, 5000	2
srv2	10.0.0.10	File server	21, 445, 8080	3
srv3	10.0.0.11	Database	3306, 5432	4
srv4	10.0.0.12	Mail and DNS	25, 53, 143	5
h3	10.0.0.3	Normal client, steady heartbeat traffic	none	6
h4 to h9	10.0.0.4 to 10.0.0.9	Normal clients, random traffic	none	7 to 12
h10	10.0.0.13	Attacker 2	none	13

The normal clients (h3 to h9) send background traffic all the time to random servers. That is why the dashboard always shows live connections, and why a block on an attacker can be compared with normal traffic that keeps flowing.

The attacks you can run (Terminal 4)

Key	Attack	What the program does
1	Illegal TCP flags	Sends packets with impossible flag combinations (NULL, XMAS, SYN+FIN).
2	Port scan	Sends connection requests to ports 1 to 40 on the server.
3	Multi-IP spoof	Sends packets that claim to come from many different IP addresses.
4	SYN connection flood	Opens about 80 new connections per second for 12 seconds.
5	Slow-and-low flows	Opens connections slowly but keeps them open, until 40 or more are held.
6	Volumetric flood	Opens 4 connections and sends a very high number of small packets for 15 seconds.
c	Coordinated attack	h1 runs the connection flood and h10 runs the volumetric flood at the same time. Expect two blocks (10.0.0.1 and 10.0.0.13).

Project files

File	What it does
controller/ids_engine.py	The IDS: the seven rules and their history windows. No network code, so it can be tested alone.
controller/ids_controller.py	The IPS: Ryu application. Handles Packet-In, installs fast-path and DROP rules, polls flow stats, handles release, writes logs.
controller/ids_rules.json	All limits, the protected list and the timeouts. Reloaded automatically.
topology/topo.py	Builds the 13-computer network and starts servers and background traffic.
apps/attack_*.py	The attack programs (flags, scan, spoof, connection flood, packet rate).
apps/background_traffic.py	Normal random traffic from h3 to h9.
tools/telemetry_panel.py	The dashboard server and web page.
tools/sdn_cli.py	Helper commands (flows, unblock, reports, tests).
t1_dataplane.sh to t4_attacker.sh	The four terminal scripts (section 9).
auto_demo.sh	Runs the whole demo with one command.
logs/	status.json, ids_events.jsonl, flow_stats.csv and controller output.

8. Questions you may be asked, with simple answers

Question	Answer
What is the difference between IDS and IPS?	An IDS detects an attack and reports it. An IPS detects it and also stops it. In this project the engine detects (IDS) and the controller installs the DROP rule (IPS).
Why SDN?	The controller sees every new connection and can change the switch's rules at any time from one place, in milliseconds. A normal switch cannot do this.
Does every packet go to the controller?	No. Only the first packet of a new connection. After it is allowed, the switch forwards the rest by itself.
Then how do you catch a flood on an already-open connection?	The controller reads the switch's packet counters every 3 seconds. That is how `packet_rate` and `concurrent_flows` work.
How does the block work?	The controller sends a FlowMod that adds a DROP rule with priority 100 (by IP) or 200 (by port). The switch drops the packets in the switch itself.
How do you know the block is really active?	The controller sends a Barrier request and waits for the reply. The time it takes is logged as "response" (a few milliseconds).
How does a blocked host get released?	The DROP rule has a 30-second idle timeout. When the attacker stops for 30 seconds the switch deletes the rule and tells the controller.
Why do you block a port for spoofing?	A spoofer changes its IP address. Blocking one IP would not stop it. Everything arrives through the same switch port, so the port is blocked.
Are normal users affected?	No. They are on the protected list and they are not matched by the DROP rules. The dashboard shows them as UP and ALLOWED during the attack.
Can the limits be changed?	Yes. Edit `controller/ids_rules.json`. The controller reloads it within 3 seconds.
What are the limits of this project?	It looks at TCP only (UDP and ICMP floods are not detected). The limits are fixed numbers, not learned. It runs on one switch. These are good "future work" points.
What could be added later?	ICMP and UDP flood detection, OpenFlow meters (rate limiting instead of blocking), automatic limits from normal traffic, and several switches.

9. One-page command sheet

Rule: start Terminal 1 first and wait for it, then Terminal 2, then 3, then 4. The project folder is `~/Desktop/sdn-ids`. Every line below is one complete command, so you can paste a whole box at once.

Step 0. Clean slate. Run in any terminal before every fresh start.

```
sudo tmux kill-server 2>/dev/null
sudo pkill -9 -f 'topology/topo'
sudo pkill -9 -f ryu-manager
sudo pkill -9 -f mnexec
sudo pkill -9 -f 'mininet:'
sudo mn -c
sudo ovs-vsctl --if-exists del-br s1
sleep 3
```

Terminal	Job	Commands	Wait until you see
T1	**Data plane** switch + 13 hosts	`cd ~/Desktop/sdn-ids` `sudo ./t1_dataplane.sh`	"13-host office LAN is live" and the `mininet>` prompt. Then leave it alone.

T2	**Control plane**
	IDS/IPS controller
		`cd ~/Desktop/sdn-ids`
		`source ~/ryu-venv/bin/activate`
		`sudo -E ./t2_controlplane.sh`
			`[SWITCH] s1 connected`. Alerts and blocks print in this terminal.

T3	**Dashboard**	`cd ~/Desktop/sdn-ids` `sudo ./t3_telemetry.sh`	Open `http://127.0.0.1:8080` and press **Ctrl+Shift+R**. Header must show "controller: up" and "switch s1: connected".
T4	**Attacker console**	`cd ~/Desktop/sdn-ids` `sudo ./t4_attacker.sh`	The menu appears. Start only when the dashboard header is green.

Terminal 4 menu keys

1	Illegal flags	**2**	Port scan	**3**	Multi-IP spoof	**4**	SYN flood
5	Slow-and-low	**6**	Volumetric flood	**c**	Coordinated h1 + h10	**7**	All six in a row
a	Switch attacker h1 / h10	**8**	Clear blocks	**9**	Show flow table	**0**	Check blocked + h3 fine
s	Start background traffic	**r**	Final report (CSV)	**q**	Quit console		

Demo order that works well

1. Show the dashboard with normal traffic (all ALLOWED, no red).
2. Press **6** in Terminal 4: a spike, then BLOCKED, then an ALERT line in the feed.
3. Press **8** to clear the block.
4. Press **c**: two attackers, two blocks (10.0.0.1 and 10.0.0.13).
5. Point at the green ALLOWED rows: normal users were not affected.
6. Press **r** for the final numbers.

Or run everything with one command. It runs the scripted attacks and keeps everything up afterwards.

```
cd ~/Desktop/sdn-ids
source ~/ryu-venv/bin/activate
sudo -E KEEP_UP=1 ./auto_demo.sh
```

Then open a second terminal for the dashboard: `cd ~/Desktop/sdn-ids` and `sudo ./t3_telemetry.sh`.

If this happens	Do this
Dashboard says "controller: down" or "switch: down"	Run Step 0, then T1, T2, T3 again, in order. (The auto demo without `KEEP_UP=1` stops everything when it finishes.)
"Found multiple Mininet shells"	Two networks are running. Run Step 0, then start exactly one T1.
Attack does nothing	Check the header is green. Press **8** and try again. Rules 5 and 6 can take up to 3 s to react.
Page looks old after an update	Restart T3 and press Ctrl+Shift+R.